Master's Degree in Electronics, Electrical Energy, and Automation

Specialization: Robotics

Academic Year 2024–2025

From the University of Montpellier





Project Report

Design and Implementation of a Digital Twin for Upper-Body Posture Prevention

by

KAIS Mehdi

DJUNAEDI Rendra Anughrah Budiputra

Supervisor: **Yassine HADDAB**

Contents

1 Introduction **3**

2 state of art **4**
2.1 Digital Twin Overview 4
2.2 Types of Digital Twins 4
2.3 Digital Twin Applications in Robotics 4
2.4 Advantages and Practical Applications of Digital Twins 4
2.5 Potential Risks and Challenges of Digital Twins 4

3 Global Architecture **5**
3.1 ESP32-WROOM Module 6
3.2 MPU6050 IMU 7
3.2.1 Calibration 7
3.2.2 Connection to ESP32 8
3.3 Webcam and Posture Detection Using MediaPipe 9
3.3.1 Landmark Detection 9
3.3.2 Movement and Angle Analysis 9
3.3.3 OpenCV Integration 10

4 Software Architecture **10**
4.1 MQTT Communication 10
4.1.1 Architecture Principle 11
4.1.2 Data Flow 11
4.1.3 Broker Deployment 11
4.1.4 Quality of Service (QoS) 11
4.2 Data Logging and Visualization with InfluxDB, Telegraf, and Grafana . . . 11
4.2.1 InfluxDB 12
4.2.2 Telegraf 12
4.2.3 Grafana 12
4.2.4 System Deployment 13

5 Digital Twin Modeling and Real-Time Synchronization **14**
5.1 Digital Twin Implementation in Blender 14
5.2 Kinematic Modeling Using Denavit–Hartenberg Parameters 14
5.3 Real-Time Synchronization 16

6 System Operation Modes and Hardware–Software Integration **17**
6.1 Hardware–Software Integration 17
6.2 Monitoring Mode 17
6.3 Simulation Mode 17

7 Streamlit Control Interface **17**
7.1 Overview 17
7.2 Interface Design 18
7.3 Database and Visualization Integration 18

8 Experimental Validation and Performance Evaluation **19**

9 Conclusion **21**

1 Introduction

Poor upper-body posture is a growing concern in modern societies, particularly due to prolonged sitting, extensive computer use, and sedentary working conditions. Incorrect posture can lead to musculoskeletal disorders, chronic pain, reduced productivity, and long-term health issues. As a result, posture monitoring and prevention systems have become an important research topic in ergonomics, healthcare, and human-centered technologies.

Recent advances in sensing technologies, computer vision, and connected systems have enabled continuous and non-intrusive monitoring of human posture. Wearable inertial sensors, vision-based pose estimation, and Internet of Things (IoT) architectures allow real-time acquisition of body motion data. However, raw data alone are often insufficient to provide intuitive feedback or comprehensive analysis of posture behavior over time.

In this context, the concept of a *digital twin* offers a powerful solution by creating a virtual representation of a physical system that evolves synchronously with real-world data. By integrating sensor measurements and vision-based information into a virtual human model, a digital twin can provide real-time visualization, posture assessment, and historical analysis. Such an approach enables not only monitoring but also simulation, replay, and potential prediction of posture-related risks.

This project focuses on the design and implementation of a digital twin dedicated to upper-body posture prevention. The proposed system combines inertial sensing, camera-based pose estimation, real-time communication, data storage, and 3D visualization to create an interactive and synchronized virtual representation of the user's posture. The objective is to provide an effective tool for posture awareness, ergonomic monitoring, and future extensions toward intelligent posture correction.

The following section presents the state of the art related to digital twins, their classifications, applications in robotics, and their advantages and challenges, providing the theoretical background necessary for the proposed system.

2 state of art

2.1 Digital Twin Overview

A digital twin is a virtual representation of a physical object, process, or system that mirrors its behavior using real-time and historical data. It enables monitoring, simulation, analysis, and optimization of performance, supporting better decision-making across industries such as manufacturing, healthcare, and urban planning [8].

2.2 Types of Digital Twins

Digital twins can be classified into five progressive levels based on their capabilities [8] :

- **Level 1 – Virtual Twin:** Static digital model used mainly for visualization and reference.
- **Level 2 – Connected Twin:** Integrates real-time sensor data for continuous monitoring.
- **Level 3 – Predictive Twin:** Uses data analytics to forecast issues and enable predictive maintenance.
- **Level 4 – Prescriptive Twin:** Applies simulations and machine learning to recommend optimal actions.
- **Level 5 – Autonomous Twin:** Operates independently, making decisions and adapting automatically without human intervention.

2.3 Digital Twin Applications in Robotics

In robotics, digital twins connect virtual design with real-world operation through high-fidelity simulations of robotic systems. They enable virtual commissioning, performance optimization, and control testing before deployment, reducing errors and development time. Real-time synchronization with industrial communication protocols ensures seamless integration with physical robots, while 3D HMIs and AI-based simulations improve visualization, training, and operator interaction. Overall, digital twins enhance the efficiency, reliability, and intelligence of modern robotic systems [9].

2.4 Advantages and Practical Applications of Digital Twins

Digital twins combine multiple technologies to create virtual replicas of systems or processes, enabling safe and cost-effective simulation, analysis, and management. They support predictive maintenance to reduce downtime, process optimization through scenario testing, faster product development via virtual prototyping, and continuous performance monitoring for proactive and informed decision-making.

2.5 Potential Risks and Challenges of Digital Twins

While digital twins offer significant advantages, their implementation is not without risks and challenges. Key considerations include [8] :

- **Initial Costs:** Implementing digital twin technology can require substantial investment, potentially limiting adoption to larger organizations.

- **Data Quality:** The accuracy and effectiveness of a digital twin depend heavily on the integrity and timeliness of data, necessitating strict data management practices.
- **Integration Challenges:** Incorporating digital twins into existing systems can be complex and labor-intensive, requiring compatibility across multiple platforms and processes.
- **Technical Expertise:** Deployment demands specialized skills in data analytics, software development, and system scaling, which can be a barrier for some organizations.

3 Global Architecture

The proposed system is based on a hybrid hardware and software architecture designed for motion detection, data transmission, storage, visualization, and digital twin simulation.

An ESP32 microcontroller coupled with an MPU6050 inertial sensor is used to detect back movements, including slouching and leaning. Additionally, a webcam captures head movements (yaw, pitch, and roll) as well as shoulder movements, leveraging the MediaPipe and OpenCV libraries.

The collected data are transmitted via the MQTT protocol to a digital twin implemented in Blender for real-time visualization. Simultaneously, Telegraf forwards the data to InfluxDB, ensuring reliable time-series storage.

InfluxDB is connected to Grafana, providing interactive dashboards for data visualization, monitoring, and analysis.

A Streamlit-based user interface enables two operating modes:

- **Simulation Mode:** allows the selection and simulation of a specific dataset within the digital twin environment.
- **Monitoring Mode:** enables real-time tracking and observation of sensor data.

This architecture provides full integration from data acquisition to visualization and analysis, supporting both offline simulations and real-time monitoring.

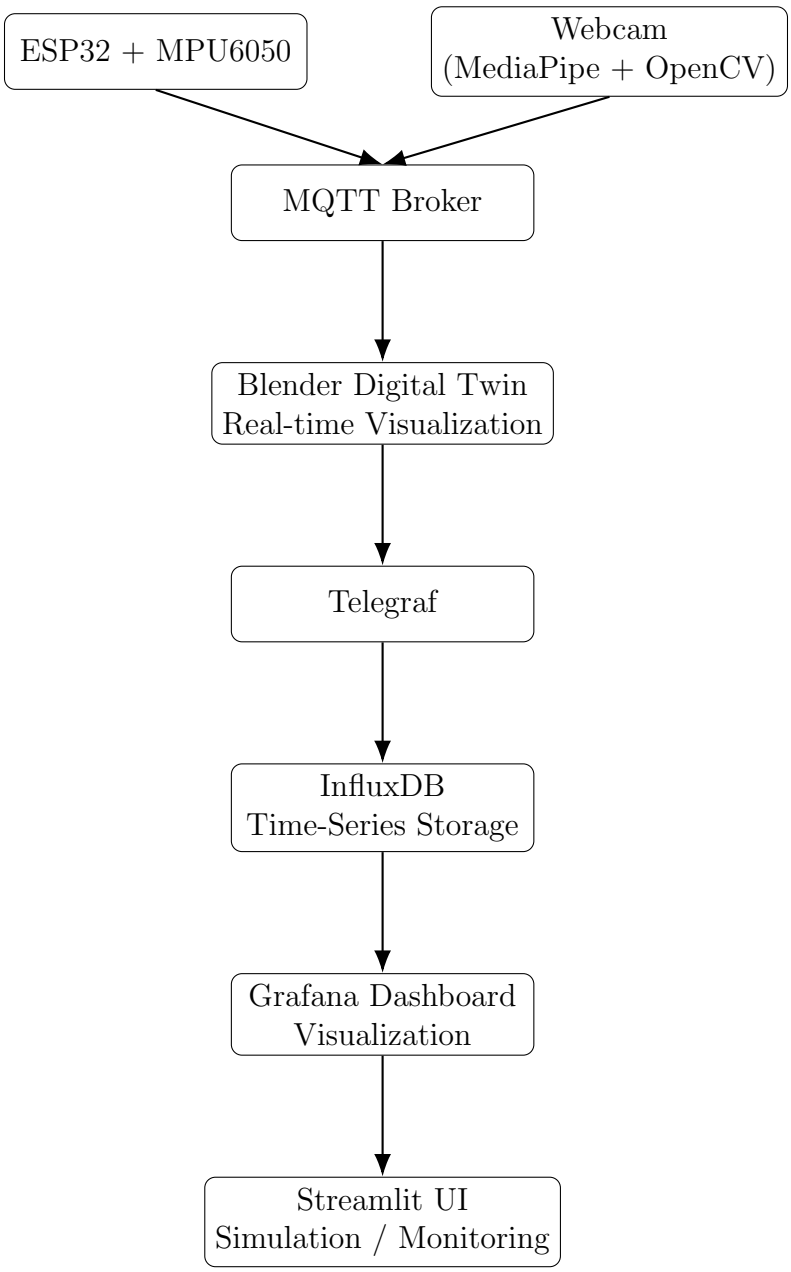


Figure 1: Global Architecture of the Posture Detection Digital Twin System

3.1 ESP32-WROOM Module

The ESP32-WROOM is a low-cost, low-power system-on-chip (SoC) microcontroller with integrated Wi-Fi and Bluetooth capabilities, widely used in IoT and embedded applications [3]. Its versatility, high processing power, and wireless connectivity make it ideal for real-time monitoring systems and digital twin projects, such as posture detection.

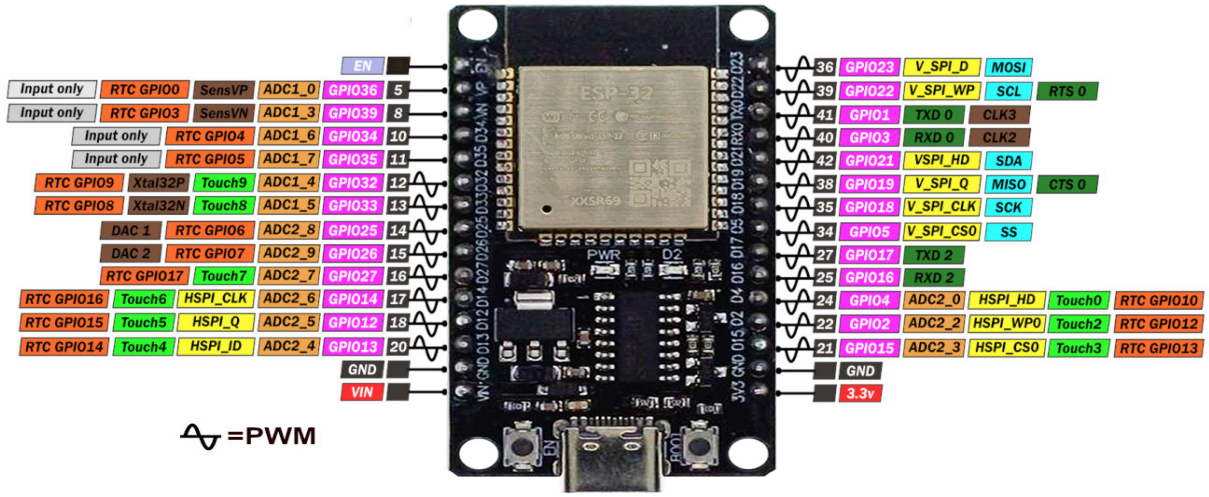


Figure 2: ESP32 [3]

Feature	**Specification**
Microcontroller	Tensilica Xtensa Dual-Core 32-bit LX6
Clock Speed	Up to 240 MHz
RAM	520 KB SRAM
Flash Memory	4 MB (typical)
Wireless Connectivity	Wi-Fi 802.11 b/g/n, Bluetooth v4.2 BR/EDR & BLE
GPIO Pins	34 programmable pins
ADC	12-bit, up to 18 channels
DAC	2 channels, 8-bit
Communication Interfaces	UART, SPI, I2C, I2S, CAN, PWM
Operating Voltage	3.0 – 3.6 V
Operating Temperature	-40°C to +85°C
Power Consumption	Low-power modes supported (e.g., deep sleep: 10 µA)
Package	38-pin QFN

Table 1: ESP32-WROOM Technical Specifications

3.2 MPU6050 IMU

The MPU6050 is a compact 6-axis inertial measurement unit (IMU) combining a 3-axis gyroscope and a 3-axis accelerometer. In this project, only the gyroscope data (rotational measurements) are used to feed the digital twin for posture tracking [10].

3.2.1 Calibration

Accurate rotational readings require a simple calibration procedure to remove zero-rate offsets. This involves keeping the sensor stationary, measuring the raw gyroscope outputs, and computing the average offset for each axis, which is then subtracted from subsequent measurements.

3.2.2 Connection to ESP32

The MPU6050 communicates with the ESP32 via the I2C interface. Typical connections are:

- VCC → 3.3 V
- GND → GND
- SDA → GPIO 21 (I2C Data)
- SCL → GPIO 22 (I2C Clock)

Pull-up resistors on SDA and SCL are recommended if not included on the breakout board. This setup allows real-time rotational data to be sent from the ESP32 to the digital twin.

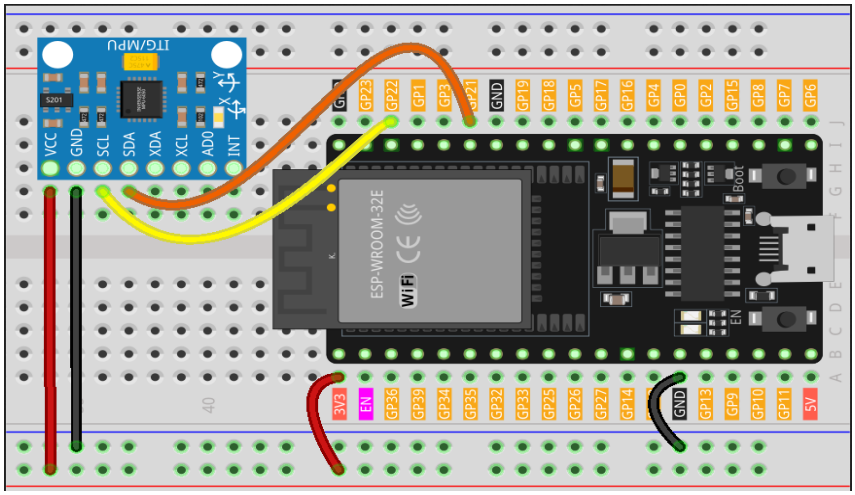


Figure 3: ESP32 with [10]

A simple enclosure was designed using **SolidWorks** to house the laptop power device (LPDC), the ESP32, and the MPU6050 in a single compact unit. The case is worn on the chest using adjustable straps to ensure stable positioning and consistent sensor measurements during motion.

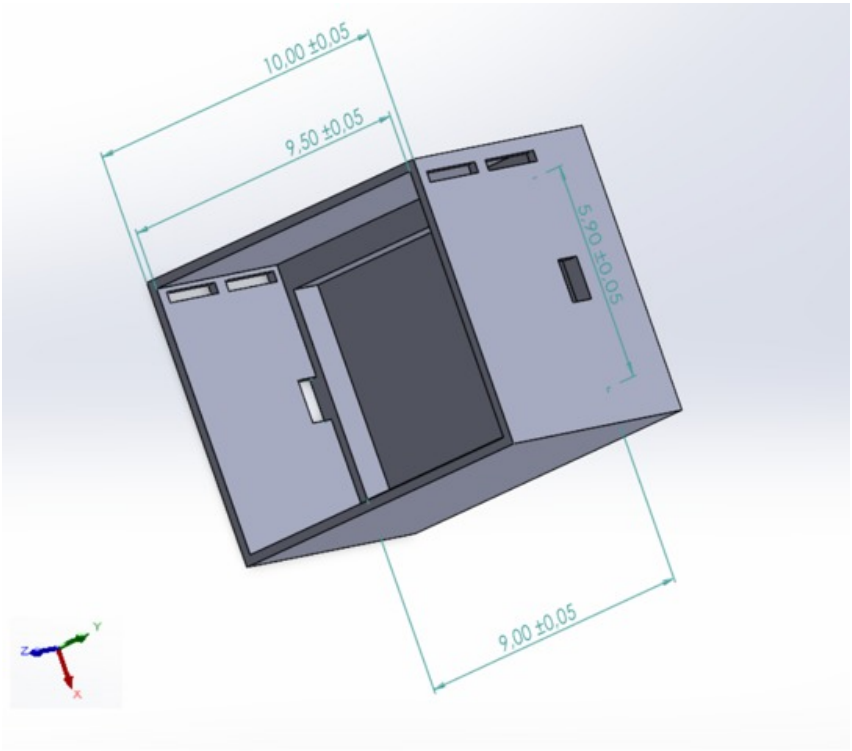


Figure 4: ESP32 Box

3.3 Webcam and Posture Detection Using MediaPipe

In this project, a standard webcam is used to capture real-time images of the user's upper body. The captured video is processed with **MediaPipe**, a framework for real-time pose estimation, combined with **OpenCV** for image capture and pre-processing.

3.3.1 Landmark Detection

MediaPipe detects key anatomical points (landmarks) such as the head, neck, shoulders, and torso. Each landmark is represented as a 3D vector (x, y, z) relative to the camera frame. By connecting landmarks, the system computes vectors representing bones or body segments, such as the vector from shoulder to elbow or neck to head [6].

3.3.2 Movement and Angle Analysis

The digital twin uses these vectors to estimate joint angles and posture deviations:

1. Tangents and directional changes between vectors are analyzed to quantify bending, slouching, or leaning.

2. The first captured frame is set as the **reference posture**, and subsequent measurements are compared to detect deviations.

These computed angles are then transmitted to the digital twin via the MQTT protocol for real-time visualization and feedback.

3.3.3 OpenCV Integration

OpenCV handles:

- Video capture from the webcam
- Pre-processing such as resizing and color conversion.
- Overlaying landmarks and vectors for visualization

This combination of OpenCV and MediaPipe allows continuous tracking of the user's posture and provides the digital twin with precise anatomical rotation data for simulation and feedback.

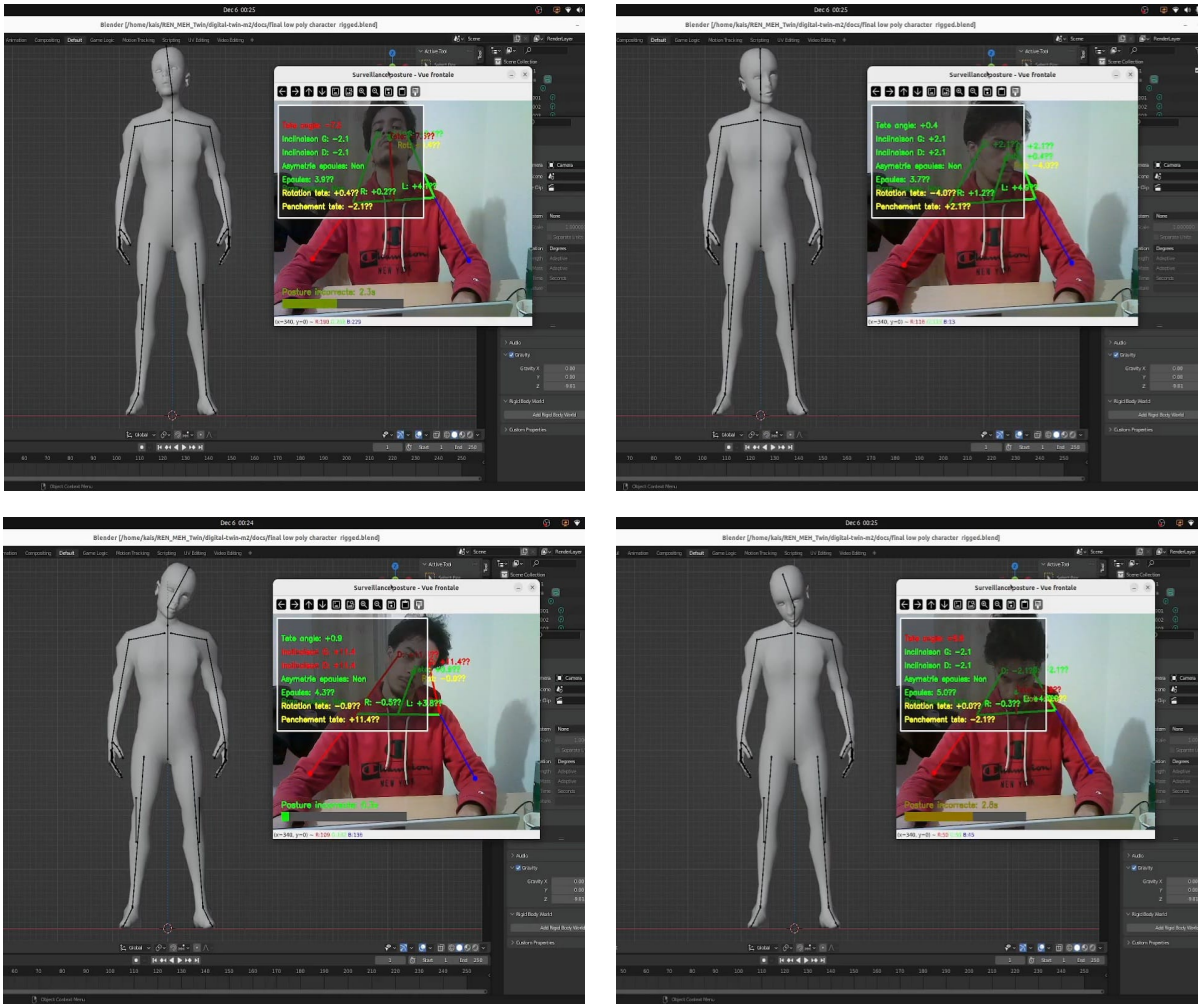


Figure 5: Results of detection posture

4 Software Architecture

4.1 MQTT Communication

To enable reliable and real-time data exchange between sensors and the digital twin, the system uses the **MQTT (Message Queuing Telemetry Transport)** protocol. MQTT is a lightweight publish/subscribe protocol widely adopted in IoT systems because of its low bandwidth consumption, low latency, and efficient communication [1].

4.1.1 Architecture Principle

MQTT follows a **broker-based architecture** where devices communicate indirectly through a central broker [1] :

- **Publishers** send data to topics,
- **Subscribers** receive data from these topics,
- the **broker** routes messages between them.

This decoupling improves scalability, modularity, and system flexibility. In this project, the Eclipse Mosquitto broker manages all message exchanges.

4.1.2 Data Flow

Each sensing module publishes its measurements:

- the **webcam + MediaPipe** module publishes computed landmarks and joint angles,
- the **ESP32 with MPU6050** transmits rotational data via Wi-Fi.

The **digital twin (Blender)**, databases, and monitoring interfaces subscribe to these topics to visualize, store, and analyze the data in real time.

4.1.3 Broker Deployment

The Mosquitto broker is deployed in a Docker container to ensure portability and easy configuration. It supports standard MQTT and WebSocket connections, logging, and persistent sessions, allowing stable and continuous operation [1].

4.1.4 Quality of Service (QoS)

MQTT defines three reliability [4] :

- **QoS 0:** fastest, no delivery guarantee,
- **QoS 1:** guaranteed delivery with possible duplicates,
- **QoS 2:** exactly-once delivery with higher overhead.

For this digital twin system, **QoS 0 or 1** is preferred to maintain low latency while ensuring sufficient reliability for real-time posture monitoring.

4.2 Data Logging and Visualization with InfluxDB, Telegraf, and Grafana

To ensure persistent storage, monitoring, and visualization of sensor data, the system integrates a time-series data stack composed of **InfluxDB**, **Telegraf**, and **Grafana**. This architecture enables continuous collection, storage, and real-time analysis of posture and motion measurements generated by the camera and the MPU6050.

4.2.1 InfluxDB

InfluxDB is a time-series database optimized for high-frequency sensor data. It efficiently stores timestamped measurements such as angles, rotations, and posture metrics. Its structure (organization, bucket, and token-based authentication) allows secure and scalable data management. In this project, a dedicated bucket named *sensors* stores all incoming measurements from the IoT devices for historical tracking and analysis [2].

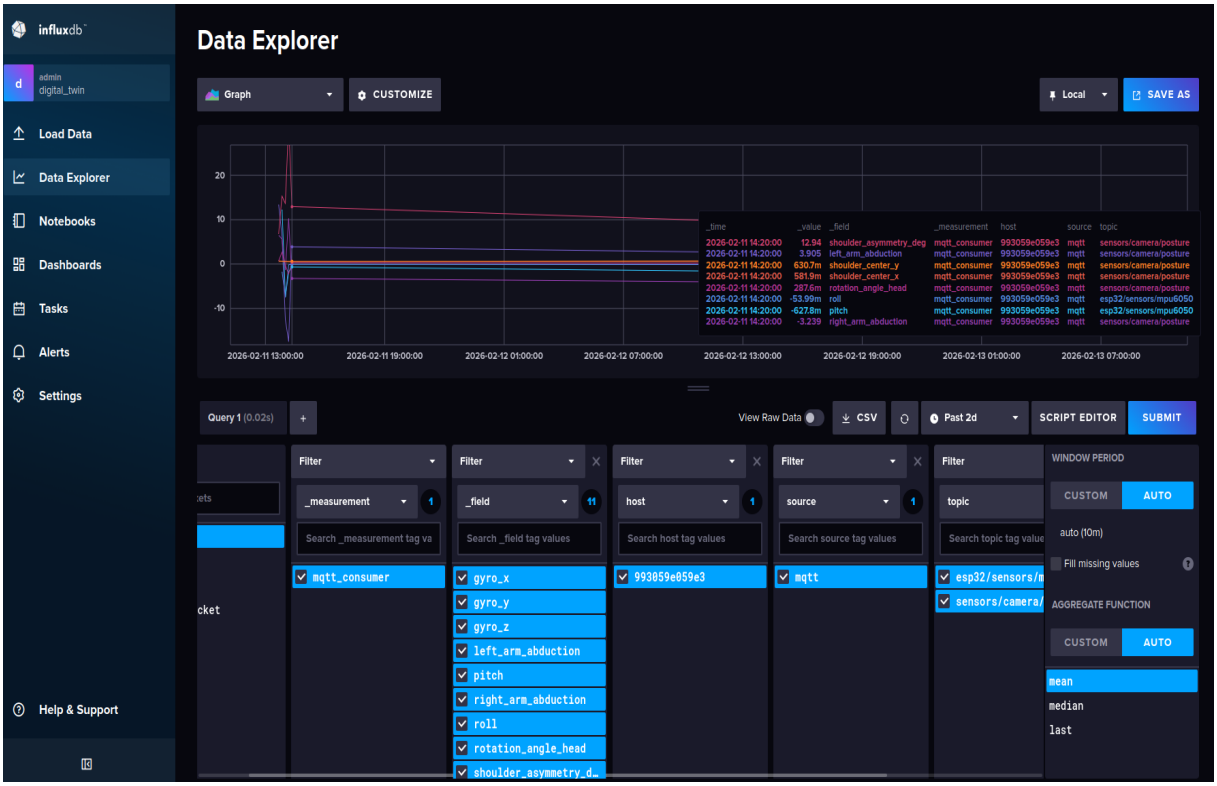


Figure 6: Grafana configuration with influxDB.

4.2.2 Telegraf

Telegraf acts as a data collection agent between MQTT and InfluxDB. It subscribes to the MQTT broker, listens to the topics published by the ESP32 and the camera modules, parses the received JSON data, and automatically forwards them to InfluxDB. This approach decouples data acquisition from storage while enabling real-time streaming. The agent operates with short sampling intervals to ensure near real-time updates of the database [2].

4.2.3 Grafana

Grafana provides the visualization layer of the system. It connects directly to InfluxDB and displays the stored measurements through dashboards, charts, and statistics. This allows users to monitor posture evolution, sensor signals, and system performance in real time. Grafana also facilitates long-term analysis by enabling comparison of historical data and detection of trends or anomalies [2].

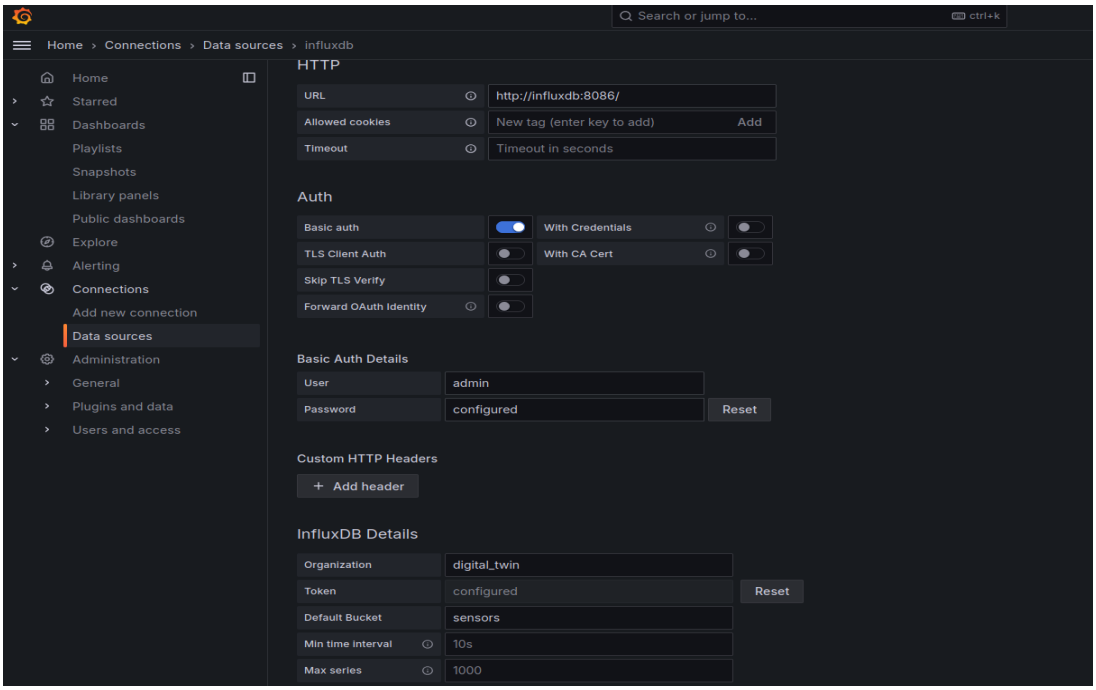


Figure 7: Grafana configuration with influxDB.

sensor data are automatically collected, stored, and visualized, providing a complete monitoring infrastructure for the digital twin system.

4.2.4 System Deployment

All three services are deployed using Docker containers to ensure portability, isolation, and easy maintenance. Containerization simplifies configuration and guarantees consistent execution across different environments. Within this pipeline:

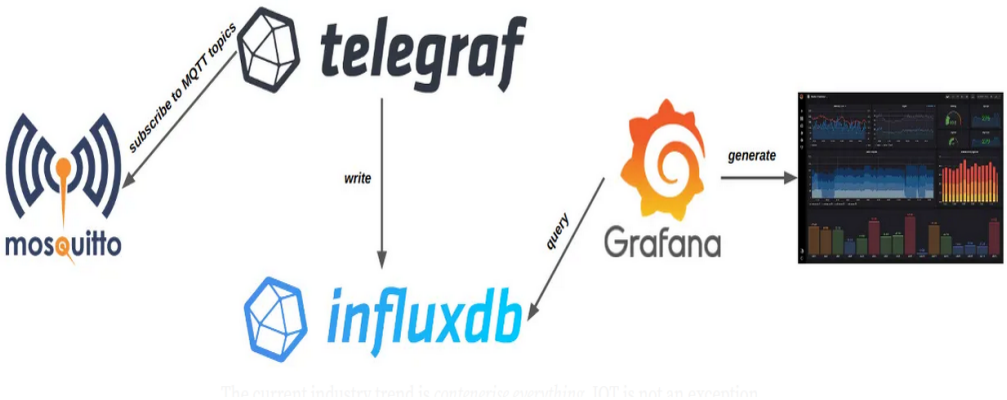


Figure 8: The resulting tech stack, all of that running inside Docker [2].

sensor data are automatically collected, stored, and visualized, providing a complete monitoring infrastructure for the digital twin system.

5 Digital Twin Modeling and Real-Time Synchronization

5.1 Digital Twin Implementation in Blender

The digital twin of the human upper body is implemented using Blender, a 3D modeling and animation environment. Blender provides a flexible platform for creating a virtual representation of the user's posture through a rigged skeletal model (armature). Each bone of the armature corresponds to an anatomical segment such as the torso, neck, and shoulders.

The model is designed to replicate human kinematics by allowing independent rotations at each joint. Real-time sensor and vision data are applied directly to these joints, enabling the avatar to mimic the user's posture dynamically. This visual feedback helps users immediately identify incorrect positions or slouching behavior.

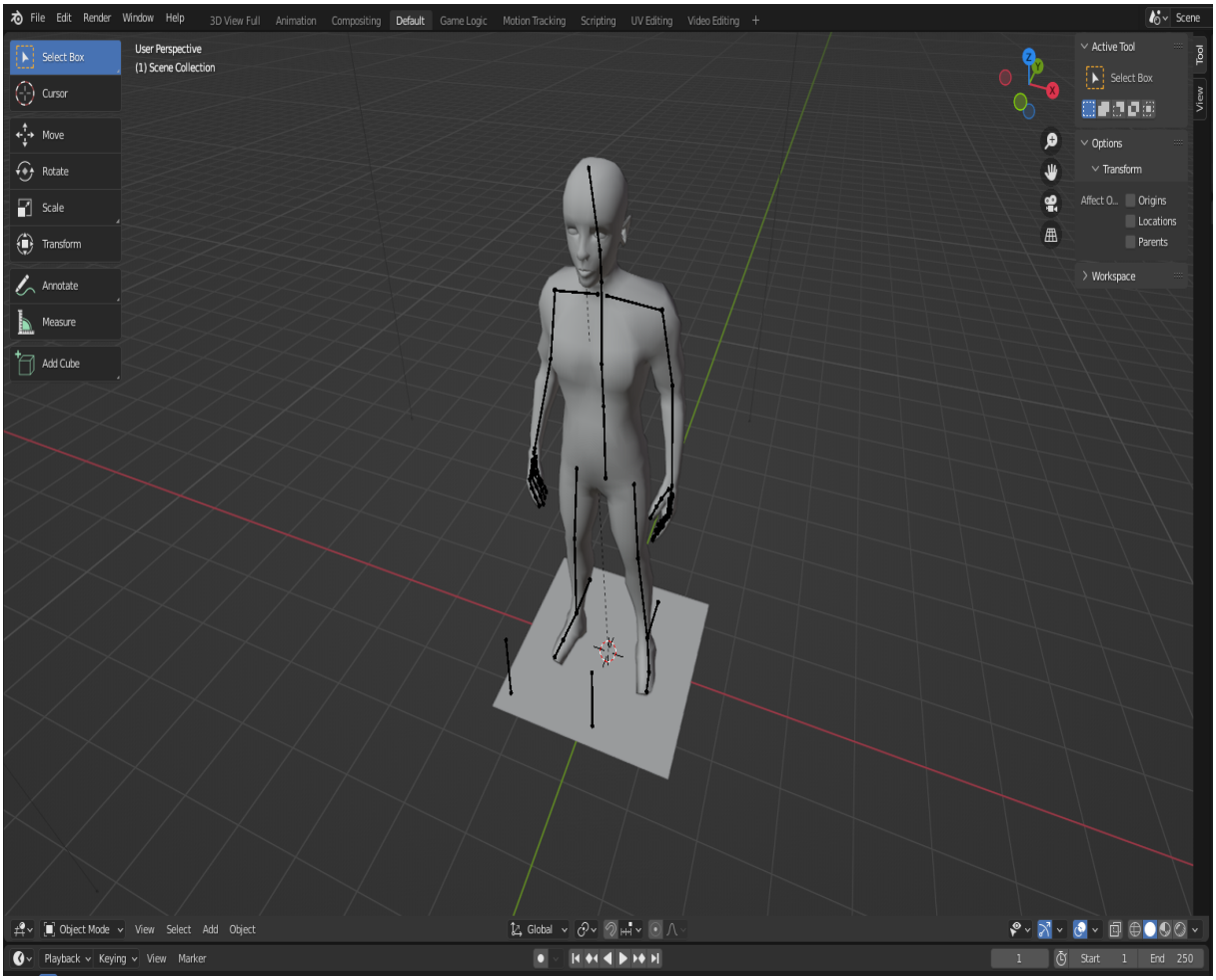


Figure 9: Digital twin on blender.

5.2 Kinematic Modeling Using Denavit–Hartenberg Parameters

To ensure consistent and physically meaningful motion, the upper body is modeled using the Denavit–Hartenberg (DH) convention. This method describes the body as a chain of rigid links connected by rotational joints.

The angles obtained from MediaPipe (camera) and the MPU6050 (gyroscope) are converted into joint rotations using DH transformations. These transformations map the measured anatomical movements to the corresponding bones of the digital twin, ensuring realistic and coherent motion.

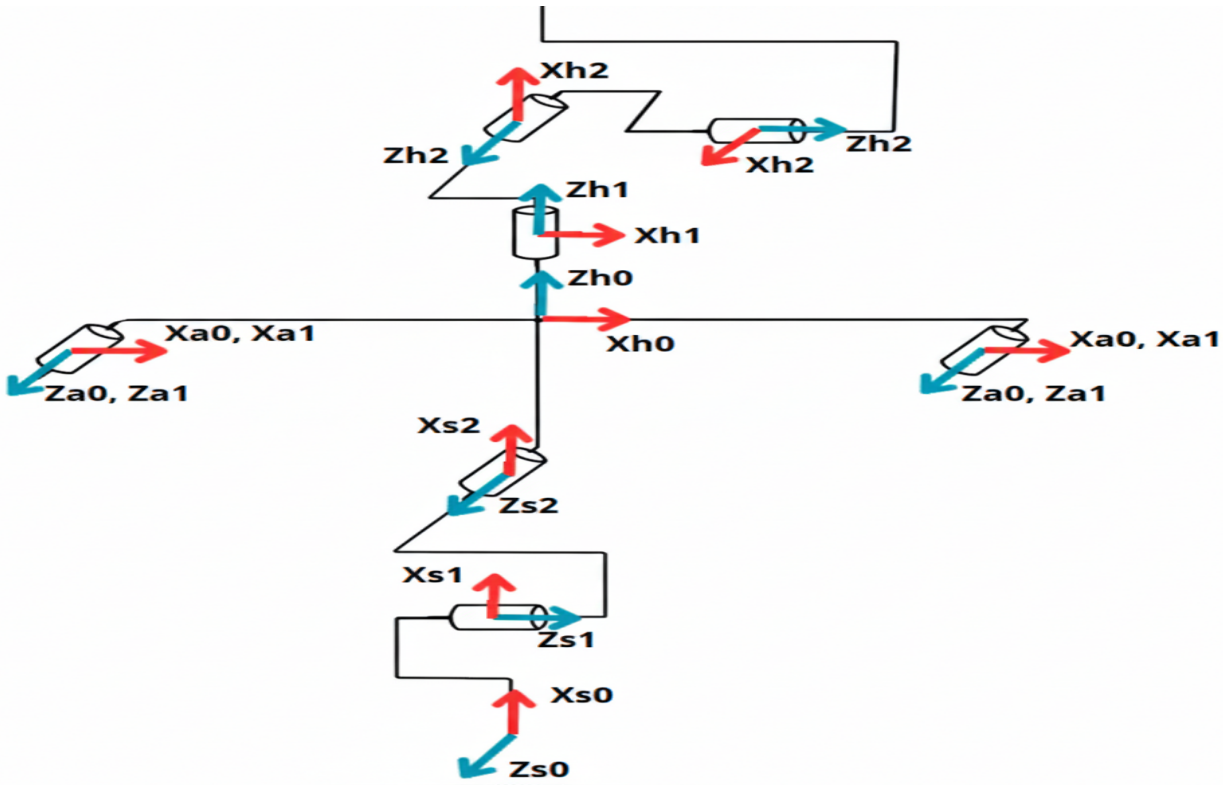


Figure 10: Joint configuration of the body

Body Modeling Description: For the choice of body movement modeling, we represented the spherical joint of the head in Blender with three degrees of freedom: yaw, pitch, and roll [7]. MediaPipe retrieves the yaw, pitch, and roll angles and applies them directly to the Denavit–Hartenberg (DH) model.

Regarding the torso (spine), it was modeled with two degrees of freedom. The MPU sensor retrieves roll and pitch angles and applies them directly to the back segment of the model.

As for the two arms, they are each modeled with one degree of freedom, since we are using a 2D camera without 3D reconstruction.

Head Denavit–Hartenberg Table

j	σ_j	α_{j-1}	a_{j-1}	θ_j	r_j
1	0	0	0	θ_1	0
2	0	$\frac{\pi}{2}$	0	$\theta_2+\frac{\pi}{2}$	0
3	0	$\frac{\pi}{2}$	0	θ_3	0

Right Arm Denavit–Hartenberg Table

j	σ_j	α_{j-1}	a_{j-1}	θ_j	r_j
1	0	0	0	θ_1	0

Left Arm Denavit–Hartenberg Table

j	σ_j	α_{j-1}	a_{j-1}	θ_j	r_j
1	0	0	0	θ_1	0

Spin Denavit–Hartenberg Table

j	σ_j	α_{j-1}	a_{j-1}	θ_j	r_j
1	0	$\frac{\pi}{2}$	0	θ_1	0
2	0	$-\frac{\pi}{2}$	0	θ_2	0

5.3 Real-Time Synchronization

Real-time synchronization is achieved through the MQTT communication layer. Sensor modules publish posture and rotation data continuously, while Blender subscribes to the corresponding topics.

The workflow operates as follows:

1. Sensors capture posture and rotation measurements,
2. data are transmitted via MQTT,
3. Blender receives updated angles,
4. the armature joints are rotated instantly.

This pipeline enables low-latency updates, allowing the digital twin to move properly in real time and mirror the user's movements almost instantaneously, ensuring smooth motion and stable synchronization. Such synchronization provides live visualization, immediate feedback, and supports monitoring, recording, and simulation modes.

Overall, the integration of Blender visualization, DH-based kinematic modeling, and MQTT streaming creates an interactive, stable, and responsive digital twin capable of accurately reproducing human posture in real time.

6 System Operation Modes and Hardware–Software Integration

6.1 Hardware–Software Integration

The system integrates embedded sensors, computer vision, communication middleware, and a digital twin within a modular architecture. Each component performs a specific task while exchanging data through the MQTT protocol.

At the hardware level, the ESP32 acquires rotational measurements from the MPU6050, and the webcam captures the user's posture. These signals are locally processed to estimate joint angles and anatomical landmarks using MediaPipe and OpenCV.

At the software level, the processed data are published to the MQTT broker and consumed by the Blender digital twin for real-time visualization, while database services store the measurements for analysis.

6.2 Monitoring Mode

The monitoring mode operates with live data acquisition. Sensor measurements are continuously transmitted through MQTT and immediately applied to the digital twin model [8].

The Blender avatar mirrors the user's posture in real time by updating the joint rotations computed from the camera and gyroscope signals. Visual cues or alerts can be generated when predefined angular limits are exceeded. This mode is primarily used for immediate posture correction and daily ergonomic monitoring.

6.3 Simulation Mode

Besides real-time monitoring, the system provides an offline **simulation mode** that replays historical measurements stored in the time-series database. Instead of live sensors, previously recorded data are reused to reproduce past posture sessions within the digital twin [8].

The process consists of retrieving timestamps and joint angles from InfluxDB, synchronizing the signals chronologically, and republishing them to the MQTT broker to emulate real-time streams. Since the digital twin subscribes only to MQTT topics, it processes replayed data exactly as live data, allowing faithful reproduction of recorded movements.

This mode enables reproducible experiments, retrospective posture analysis, and adjustable replay speed for faster or slower simulations while preserving temporal consistency.

7 Streamlit Control Interface

7.1 Overview

To provide an intuitive and centralized supervision layer for the digital twin system, a graphical control interface was developed using **Streamlit**. This lightweight Python framework enables the rapid creation of interactive web applications while remaining fully compatible with the existing data-processing scripts and backend services [5].

The interface acts as a *control center* that connects all software and hardware components, including the MQTT broker, the sensing modules, the database services, and the

Blender digital twin. It allows users to configure parameters, start or stop services, and monitor the system from a single dashboard without requiring command-line interaction.

7.2 Interface Design

The dashboard is organized into modular sections to improve clarity and usability:

- a sidebar dedicated to system configuration and service management,
- status indicators showing the availability of MQTT, database, and visualization services,
- monitoring panels displaying live sensor values,
- interactive plots for real-time signal visualization,

This structured layout simplifies system supervision and provides immediate feedback about the operational state of each component.

7.3 Database and Visualization Integration

The interface communicates directly with InfluxDB to retrieve time-series sensor measurements. It can query recent or historical data and display them as dynamic charts inside the application.

For advanced analytics and professional dashboards, Grafana panels are embedded within the interface, enabling detailed visualization of long-term trends and performance metrics.

Additionally, the interface interacts with Blender by ensuring that the digital twin receives the required data streams through MQTT, allowing smooth and continuous avatar animation.

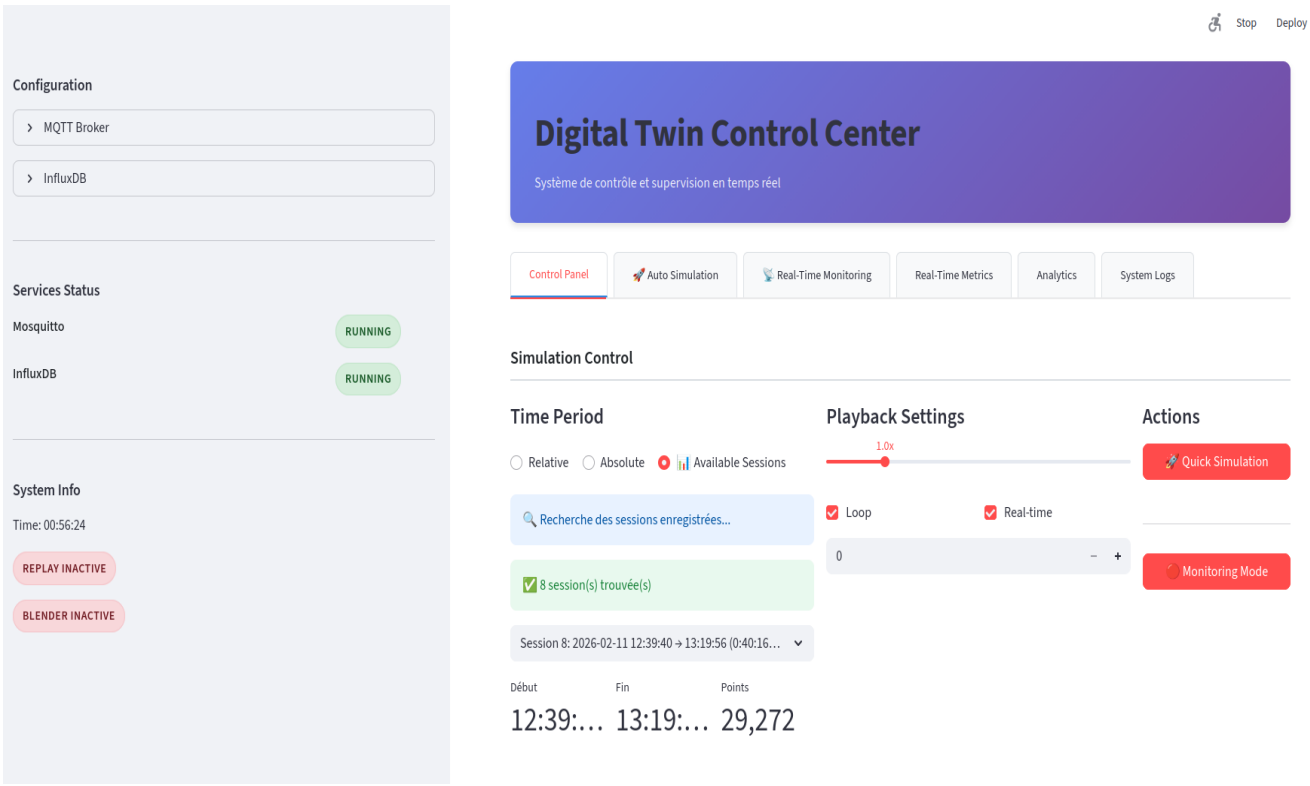


Figure 11: Streamlit interface.

8 Experimental Validation and Performance Evaluation

After the complete implementation of the prototype, experimental tests were carried out to validate posture detection accuracy, real-time synchronization, and long-term monitoring stability.

Figure 12 presents the digital twin response for different user positions, including upright posture, forward slouching, and lateral leaning. The virtual model accurately reproduces torso inclination measured by the MPU6050 and head/shoulder rotations estimated using MediaPipe.

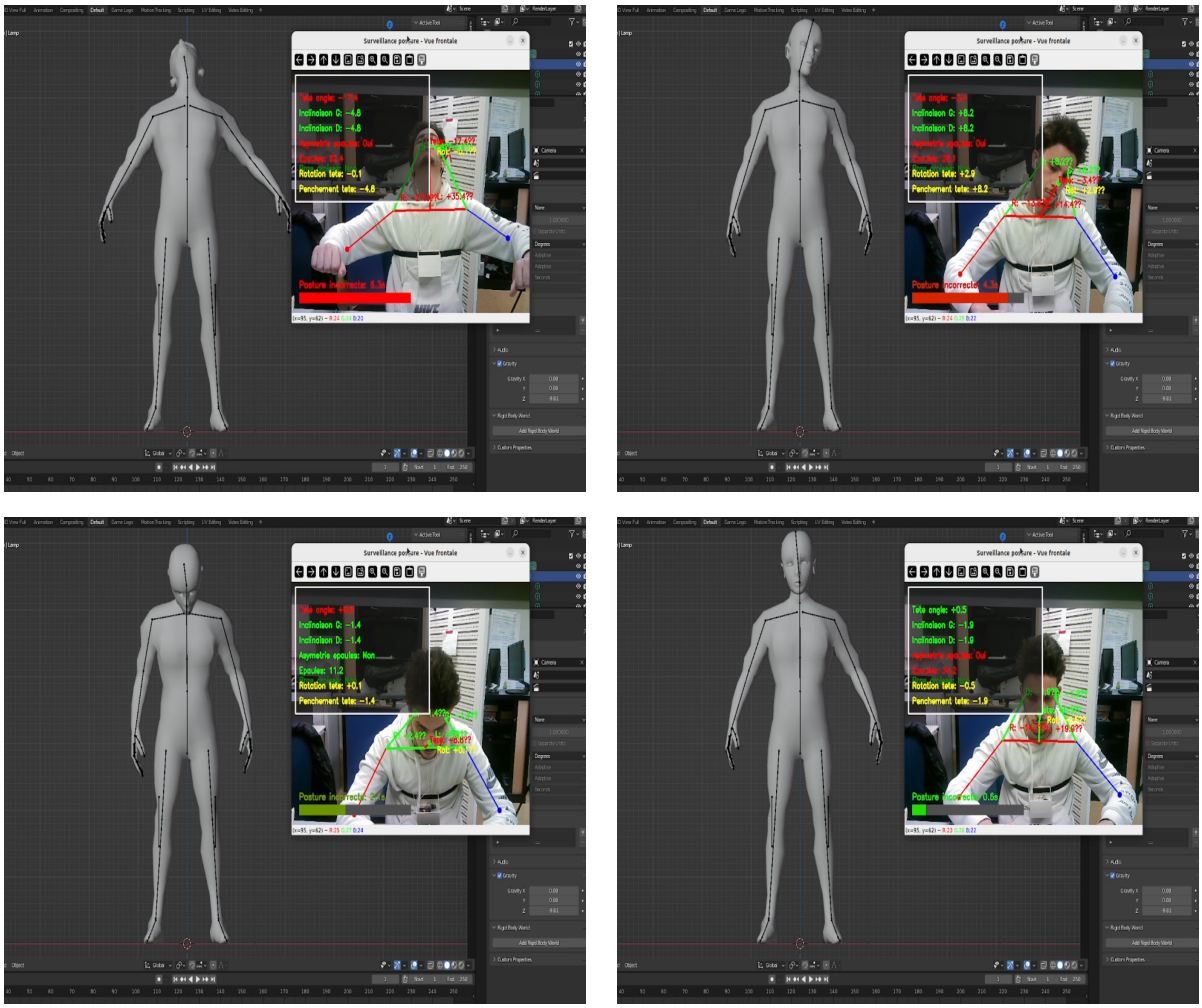


Figure 12: Digital twin response for different user postures

A one-hour continuous test was then performed in Monitoring Mode to evaluate system robustness. Sensor data were transmitted via MQTT, stored in InfluxDB, and visualized in Grafana. As shown in Figure 13, the signals remain stable over time, with consistent sampling and no major communication loss. Posture variations and corrections are clearly observable in the time-series plots.

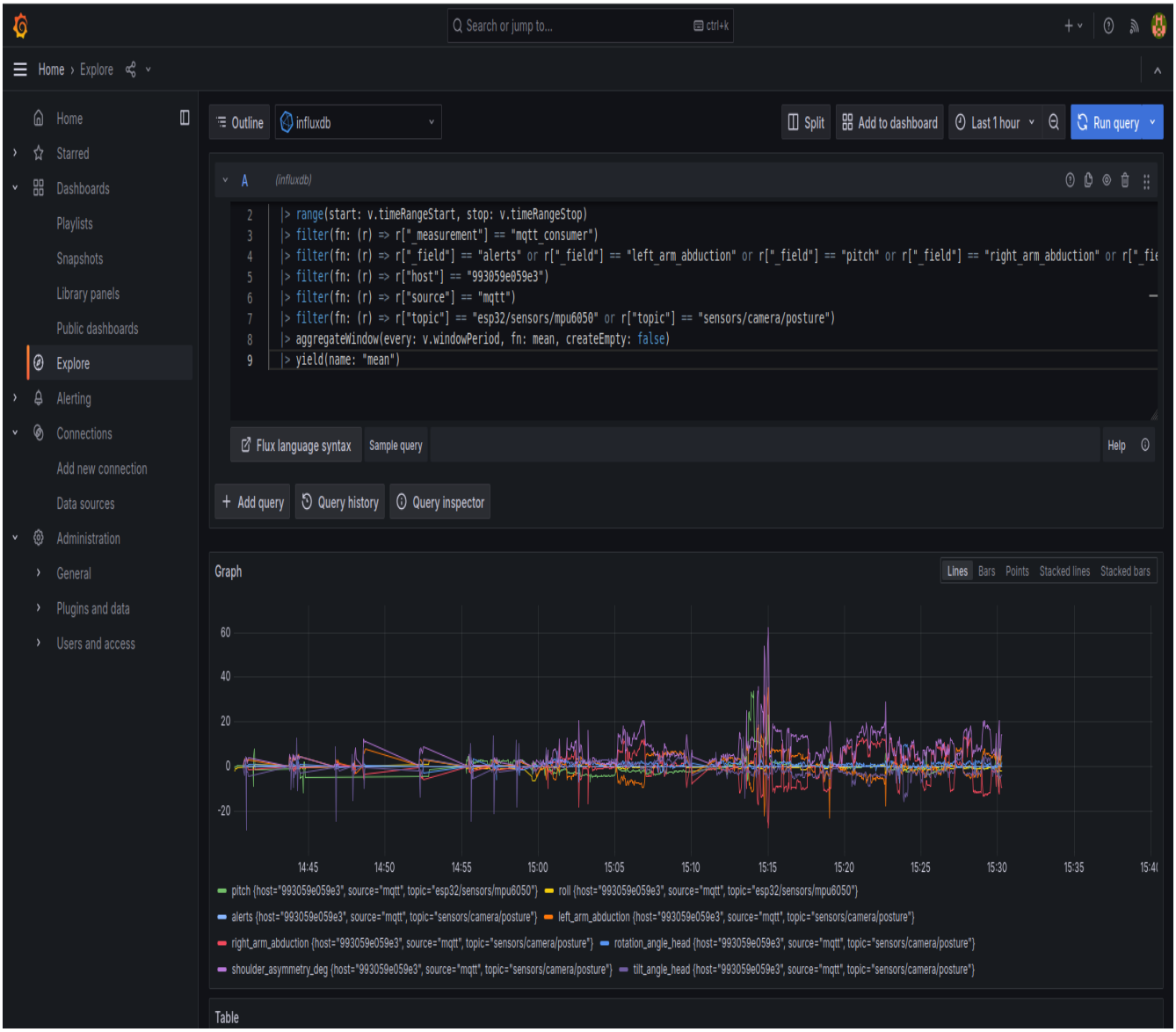


Figure 13: One-hour monitoring session in Grafana

Visualization of a one-hour digital twin monitoring session in Grafana dedicated to posture tracking. Multiple sensor signals are displayed over time, including head pitch, roll and tilt, shoulder asymmetry, as well as joint-specific movements such as left and right arm abduction. The data are acquired from an ESP32 device equipped with an MPU6050 sensor and synchronized with camera-based posture measurements. The aggregated window presents mean values over the selected time interval, enabling real-time analysis and the identification of posture variations. These results demonstrate that the system provides accurate posture tracking, reliable real-time synchronization, and effective monitoring of ergonomic parameters.

9 Conclusion

This project presented the design and implementation of a digital twin system for real-time posture monitoring. The proposed architecture integrates embedded sensors (ESP32 and MPU6050), computer vision (MediaPipe and OpenCV), MQTT communication, time-series data management (InfluxDB), visualization tools (Grafana), and a 3D digital twin developed in Blender.

The experimental results demonstrate that the system accurately reproduces upper-body movements with low latency and stable synchronization. The hybrid approach combining gyroscope measurements and vision-based estimation ensures reliable posture tracking, while the Docker-based data pipeline guarantees robust storage and visualization over long monitoring sessions.

In addition to real-time monitoring, the implementation of a simulation mode enables replay and analysis of recorded sessions, increasing the flexibility of the system. Overall, the developed solution validates the feasibility of a connected digital twin for ergonomic supervision and human posture analysis, with potential extensions toward predictive analytics and intelligent posture correction.

References

[1] AranaCorp. Mqtt and mosquitto: Guide for iot communication, 2021.

[2] Medium IoT Community. Using mqtt with influxdb and telegraf for sensor data, 2021.

[3] DOIT Electronics. *ESP32-WROOM Technical Datasheet*, 2021.

[4] HiveMQ. Mqtt quality of service levels explained, 2020.

[5] Streamlit Inc. Streamlit official documentation, 2023.

[6] Innovatianna. Mediapipe pose estimation for posture detection, 2022.

[7] Industrial Robotics Journal. Denavit–hartenberg modeling for human motion in robotics, 2022.

[8] NVIDIA. Digital twin overview and applications. 2023.

[9] John Smith and Alice Lee. Embedding digital twins in robotic systems. *Robotics and Automation Letters*, 8(4):2301–2310, 2023.

[10] SunFounder. *MPU6050 6-Axis Gyroscope/Accelerometer Module Documentation*, 2020.